FRIENDFLOW

Product Requirements Document

Consent-based friend location sharing, live maps & distance comparison

Prepared for implementation in Antigravity

React + Vite • Firebase • Capacitor • Capgo Background Geolocation • Leaflet/OpenStreetMap

Implementation specification • Privacy-first • Mobile-first

0. How to use this PRD

Give this document to Antigravity as the source of truth. Ask the agent to inspect the existing repository first, preserve working code, then implement in phases. Do not let it replace the project wholesale without explaining migrations and risks.

Product boundary: FriendFlow is a consent-based location-sharing tool for people who knowingly opt in. It must never track someone secretly, bypass operating-system permissions, or imply that browser permission alone enables background tracking.

1. Product vision

Build a polished, mobile-first friend tracker that lets registered users discover one another by username, establish mutual friendships, explicitly share their location, view only authorized friends on a map, and compare straight-line distance between two friends. When a user needs location sharing to continue with the web page closed, the location owner must use an installed native Android companion/wrapper with native background GPS. The website remains the viewer dashboard.

2. Goals and non-goals

Goals

- Secure email/password sign-up, login, logout, and profile management.
- Username-based user discovery without exposing coordinates in search results.
- Friend requests with accept, reject, cancel, and remove actions.
- Location sharing disabled by default; user starts and stops it deliberately.
- Leaflet map with authorized friends' latest shared locations and freshness indicators.
- Select Friend A and Friend B to calculate straight-line distance in kilometers/meters.
- Optional road route and ETA via a routing provider, with usage limits and fallback states.
- Background GPS through Capgo in a native Capacitor build, with a persistent notification and clear OS permission flow.
- Responsive viewer website; PWA install is optional and does not claim to provide background GPS by itself.
- Firestore Security Rules, validation, tests, graceful permission/network failures, and clear data deletion controls.

Non-goals for MVP

- Secret or non-consensual tracking, stealth mode, or bypassing OS permissions.
- Guaranteed GPS updates after Android force-stop, device shutdown, revoked permission, no GPS signal, or no network.
- Continuous background tracking from a closed ordinary browser tab.
- Chat, social feed, public leaderboards, ads, or public sharing links.
- Unlimited use of community map tiles or public routing demo servers.

3. Target users and primary journeys

Journey A — Join FriendFlow

– User registers with email/password.

– User chooses a unique username and display name.

– User lands on the dashboard with location sharing OFF.

– A clear onboarding explanation states what is shared, with whom, and when.

Journey B — Become friends

– Search a username; search results show profile basics only.

– Send a friend request.

– Recipient accepts or rejects it.

– Only accepted friends appear in the sharing/friends list; rules enforce permissions server-side.

Journey C — Share live location

– User opens Location Sharing settings and sees the sharing audience.

– User explicitly taps Start sharing and reviews OS permission prompts.

– Native tracker starts and displays a persistent foreground-service notification on Android.

– Latest coordinates and timestamp are uploaded securely; the map displays authorized updates.

– User can stop sharing at any time; tracker stops and the stored current-location document is deleted or made inaccessible immediately.

Journey D — Compare two friends

– Choose Friend A and Friend B from accepted friends with fresh shared locations.

– Show direct distance, GPS accuracy, and last-updated time.

– Optionally request road distance and ETA; label it separately from straight-line distance.

– If either location is unavailable or stale, explain why and do not invent values.

4. Functional requirements

ID	Requirement	Acceptance intent
AUTH-01	Users can register, log in, log out, and reset passwords using Firebase Authentication.	Covered by tests and verified in UI
AUTH-02	Require authentication for all private data and all location actions.	Covered by tests and verified in UI
PROF-01	Each profile has uid, username (unique normalized key), displayName, optional avatarUrl, createdAt, and sharingEnabled=false by default.	Covered by tests and verified in UI
FR-01	Users can search by exact/prefix username without returning coordinates or private fields.	Covered by tests and verified in UI
FR-02	Friend request state is pending/accepted/rejected/cancelled; prevent duplicate requests and self-friending.	Covered by tests and verified in UI
FR-03	Only accepted friendships grant friend-location read eligibility.	Covered by tests and verified in UI
LOC-01	Location sharing is explicit, revocable, visible, and off by default.	Covered by tests and verified in UI
LOC-02	Use native background GPS in the Android app/wrapper; web fallback is foreground-only and clearly labeled.	Covered by tests and verified in UI
LOC-03	Store latitude, longitude, accuracyMeters, capturedAt, receivedAt, and source/platform where appropriate.	Covered by tests and verified in UI
LOC-04	Reject invalid coordinates, unreasonable payloads, and stale/out-of-order writes where enforceable.	Covered by tests and verified in UI
LOC-05	Stop sharing stops native updates, cancels upload work, updates sharing state, and deletes/revokes the last location.	Covered by tests and verified in UI
LOC-06	Show freshness: Live/recent/stale/offline based on configurable thresholds; never label an old location as live.	Covered by tests and verified in UI
MAP-01	Show authorized friends as markers with display name, freshness, and accuracy when tapped.	Covered by tests and verified in UI
MAP-02	Map must include visible OpenStreetMap attribution if OSM tiles are used.	Covered by tests and verified in UI
DIST-01	Calculate Haversine straight-line distance client-side from two authorized, fresh locations.	Covered by tests and verified in UI
DIST-02	Optional road route uses OSRM or another configured provider; show a clear error/fallback if unavailable.	Covered by tests and verified in UI
PRIV-01	Users can see and control sharing state and sharing audience.	Covered by tests and verified in UI
PRIV-02	Friend removal/revocation immediately removes future access and clears UI cache/listeners.	Covered by tests and verified in UI
PRIV-03	Provide account deletion and location data deletion flow.	Covered by tests and verified in UI
OPS-01	Handle denied permissions, disabled location services, weak accuracy, no network, app termination, and Firebase errors.	Covered by tests and verified in UI
OPS-02	Use rate limits/throttling and avoid writing every GPS sample unnecessarily.	Covered by tests and verified in UI

5. UI/UX design system

Visual direction: calm premium utility

- Use a clean, high-trust interface—not a surveillance dashboard. The map is the primary canvas.

- Palette: deep forest #123B35, teal #167D70, mint #DDF5E9, warm off-white #F7F8F5, slate text #2B3441, danger #C43D4B. Reserve bright colors for meaningful statuses only.
- Typography: Inter or system sans; clear hierarchy; 8px spacing scale; rounded 14–20px cards; restrained shadows; no excessive glass effects over map labels.
- Mobile-first responsive layout. On mobile, map occupies most of the screen with a bottom sheet for friends and distance comparison. Desktop uses map + right sidebar.
- Accessible contrast, keyboard navigation, focus rings, reduced-motion support, semantic labels, and touch targets of at least 44px.
- Use skeletons while loading, empty states with next actions, toasts for completed actions, and inline errors for permission/network failures.
- Use Lucide icons consistently. Avoid decorative fake metrics, fake friend locations, or hard-coded online states.

Screens and key UI states

Landing / onboarding — Concise value proposition, privacy-first explanation, Sign in/Create account, no map before login.

Register / login — Simple forms, validation, password reset, loading state, accessible errors.

Dashboard / map — Map, search field, sharing status chip, location freshness legend, recenter button, friends drawer, compare-distance action.

Friends — Search, sent requests, received requests, accepted friends, remove friend confirmation.

Friend details — Display name, last updated, approximate accuracy, distance from me if allowed, open directions; no hidden coordinates exposed in profile.

Compare distance — Two selectors, direct distance card, road distance/ETA optional, freshness/accuracy labels, swap selections, empty/stale states.

Location sharing — Off/On state, audience summary, permission checklist, battery/network note, Start/Stop button, persistent notification explanation.

Privacy & settings — Manage sessions, revoke sharing, delete location history/current location, account deletion, privacy policy.

Offline/error states — No permission, location services disabled, no accepted friends, stale location, offline, Firebase failure, map tile failure.

6. Suggested app layout

```
src/
  app/ (App.jsx, routes.jsx, AuthProvider.jsx)
  components/ (AppShell, MapView, FriendMarker, FriendCard,
    FriendSearch, FriendRequestList, SharingStatus,
    SharingConsentDialog, DistanceComparePanel, EmptyState, ErrorBoundary)
  pages/ (LandingPage, LoginPage, RegisterPage, DashboardPage,
    FriendsPage, SettingsPage)
  hooks/ (useAuth, useAcceptedFriends, useFriendRequests,
    useAuthorizedFriendLocations, useForegroundLocation)
  services/ (authService, profileService, friendshipService,
    locationService, routingService)
  lib/ (firebase, distance, validators)
  styles/ (tokens.css, globals.css)
  tests/ (distance.test, friendship.rules.test, location.rules.test)
android/ (native Capacitor project and plugin configuration)
firestore.rules
firestore.indexes.json
.env.example
README.md
```

7. Data model

Collections

```
users/{uid}
  uid, username, usernameLower, displayName, avatarUrl?
  sharingEnabled: boolean (default false)
  sharingAudience: "accepted_friends" (MVP)
  createdAt, updatedAt

friendRequests/{requestId}
  fromUid, toUid, status: pending|accepted|rejected|cancelled
  createdAt, updatedAt

friendships/{pairId}
  userA, userB (sorted UIDs)
  status: accepted
  createdAt, updatedAt

users/{uid}/location/current
  latitude, longitude, accuracyMeters
  capturedAt, receivedAt, platform
  // Owner writes. Eligible accepted friends may read only while sharing is on.
```

Keep profile data separate from precise coordinates. Never return location fields from username search. Use server timestamps for receivedAt and validate payloads. For stronger trust boundaries, use a trusted Cloud Function to accept requests and create canonical friendship records; do not let users write arbitrary accepted friendship records.

8. Security and privacy requirements

- Deny by default in Firestore Security Rules; no open test-mode rules in production.
- Location read rule checks authentication, owner sharingEnabled=true, and accepted friendship between viewer and owner.
- Location writes can only be made by the authenticated owner (or a narrowly scoped trusted service); validate numeric ranges and document ID.
- Friend request creation must prevent self-requests and forged sender IDs. Acceptance must be authorized only for the recipient.
- Canonical friendship state must not be user-forgeable. Prefer callable Cloud Functions / trusted backend transaction for accepting requests and reciprocal relationship records.
- Account/profile lookup must not expose location, private email, or sensitive fields.
- Stop sharing must immediately update sharing state and stop native tracking; deletion is best effort, but rules must deny reads as soon as sharing is false.
- Use Firebase App Check where appropriate, App Check enforcement after validation, abuse protection, and rate limiting for requests/location writes.
- Do not store a location history by default. MVP stores only the latest location. If history is added later, make it opt-in with retention controls.
- Do not log exact coordinates in analytics, console logs, crash reports, or user-facing URLs.
- Document that app force-stop, permission revocation, no signal, battery restrictions, and OS scheduling can interrupt tracking.

9. Background GPS architecture

Use React inside Capacitor for the installed Android tracking client. Use @capgo/background-geolocation with a version matching the Capacitor major version. The native plugin must be started only after a clear user action and appropriate permission flow. Keep an ongoing Android notification while sharing.

- Detect platform: browser build uses foreground-only Geolocation API; native Android build may enable Capgo background tracking.

- Show a clear permission explainer before invoking OS permission prompts; request only permissions needed for the chosen features.
- Do not claim that a normal closed browser tab can track GPS. The website can view locations, while a native installed client supplies background updates.
- Implement persistent offline queue separately if needed: local durable storage, bounded queue, retry with backoff, deduplicate by event ID, discard obsolete samples according to retention policy, and upload only after authentication is revalidated.
- On sign-out, revoke or stop tracking before clearing credentials. On sharing off, stop watcher, cancel upload retries, and delete/revoke current location.
- Test on real Android devices across screen lock, background, swipe-away, force-stop, reboot, permission changes, battery saver, and network loss.
- Do not promise tracking after Android force-stop, phone power-off, revoked permissions, or denied location services.

10. Map, distance and routing

- Use Leaflet + React Leaflet. Use OpenStreetMap-derived tiles with visible attribution and respect the chosen tile provider's usage policy.
- Never bulk download public OSM tiles or assume the community tile service is unlimited. Keep tile URL configurable and document a production tile-provider migration path.
- Render markers only from authorized friend location documents. Do not include sample coordinates as if they were real people.
- Distance: use Haversine formula; label it 'Straight-line distance'. Round to 2 decimal km or nearest 10 m depending on distance.
- Show location timestamp and horizontal accuracy. If location exceeds configurable freshness threshold (e.g. 2 minutes), label it 'Last seen' or 'Stale', not 'Live'.
- Road distance/ETA is optional; route through OSRM public demo only for development/light testing. For production, choose a hosted provider or self-host OSRM and obey terms/rate limits.
- If only one friend has a valid location, center on that friend; if none, show a neutral map onboarding state, not a fabricated location.

11. Performance and reliability

- Throttle location uploads by time and/or distance; initial default can be 25–50 metres with a configurable minimum interval.
- Use Firestore listeners only for the currently accepted friend set and unsubscribe when users sign out or remove friends.
- Avoid reading all users or all locations. Use indexed queries and pagination for friend search/requests.
- Use lazy-loaded map code and markers; cluster markers if friend count grows.
- Handle Firestore permission-denied errors by clearing location from the UI immediately.
- Keep current-location document writes modest to control battery, bandwidth, and database cost.

12. Testing and acceptance criteria

- New account has sharing disabled and no location document.
- User A cannot read User B location without an accepted friendship.
- User A cannot read User B location if B turns sharing off.
- Rejecting or removing a friendship prevents future reads and removes the marker.
- Search results never contain coordinates or precise location data.
- Invalid latitude/longitude and unauthenticated writes are denied.
- Friend A/B distance matches known Haversine test cases and is labeled straight-line.

- Stale timestamps show stale/last-seen UI rather than live status.
- Permission denial and disabled device location produce actionable UI.
- Native Android test demonstrates updates with screen locked and app backgrounded under normal OS conditions.
- Offline queue (if implemented) uploads after reconnect, deduplicates events, and stops after sharing is disabled.
- Sharing stop action stops the native watcher and prevents subsequent location reads.
- Mobile UI works at 360px width; desktop layout is responsive; keyboard focus and contrast are verified.

13. GitHub repositories and documentation

Capgo Background Geolocation — https://github.com/Cap-go/capacitor-background-geolocation
Primary native background GPS plugin. Match plugin major version to Capacitor major; verify platform setup and permission requirements.

Capacitor Community Background Geolocation — https://github.com/capacitor-community/background-geolocation
Alternative lightweight plugin for background location; compare maintenance, accuracy, and current Capacitor compatibility.

React Leaflet — https://github.com/PaulLeCam/react-leaflet
React bindings for Leaflet maps.

Leaflet — https://github.com/Leaflet/Leaflet
Core open-source interactive map library.

Firebase JS SDK — https://github.com/firebase/firebase-js-sdk
Official Firebase JavaScript SDK.

Firebase quickstarts — https://github.com/firebase/quickstart-js
Examples for Firebase web features and authentication.

Capacitor — https://github.com/ionic-team/capacitor
Native runtime for packaging the React app as Android/iOS.

OSRM — https://github.com/Project-OSRM/osrm-backend
Optional road-routing engine; public demo server is not for unlimited production use.

14. Implementation phases

Phase 0 — Audit: Inspect current repository, package manager, routes, UI, Firebase setup, existing features, and Capacitor version. Report risks and plan before editing.

Phase 1 — Foundation: Design tokens, responsive shell, auth pages, Firebase initialization, environment example, route protection.

Phase 2 — Social graph: Profiles, username search, request lifecycle, canonical accepted friendship records, rules and tests.

Phase 3 — Map and distance: Leaflet map, authorized markers, freshness states, Haversine utility, two-friend comparison panel.

Phase 4 — Sharing controls: Foreground web location, consent UI, start/stop controls, current-location writes, sharing-off revocation.

Phase 5 — Native background GPS: Capacitor Android integration, compatible Capgo plugin, Android permission/notification configuration, native upload/queue strategy, real-device tests.

Phase 6 — Hardening: Rules emulator tests, App Check, error states, cost/performance checks, privacy policy, README, deployment instructions.

15. Definition of done

- All MVP flows work end-to-end with two or more real test accounts.
- No fake location markers or mock online states remain in production.

- Firestore rules are committed and tested using the Firebase Emulator Suite.
- Background GPS is tested in the native Android build; browser-only limitations are clearly stated in UI/docs.
- No secrets are committed; .env.example documents required variables.
- README includes local setup, Firebase setup, Android build/sync, permissions, deployment, security testing, and known limitations.
- Responsive accessibility and mobile performance checks pass.
- User can stop sharing and see the sharing status change immediately.

16. Copy/paste implementation instruction for Antigravity

```
You are the senior full-stack engineer implementing FriendFlow from this PRD. First inspect the existing re
pository and report its framework, package manager, routes, Firebase setup, and Capacitor version. Do not o
verwrite working code blindly. Implement in the PRD phase order, making small verifiable changes. Use React
 + Vite unless the repository already uses a suitable React framework; Firebase Auth and Firestore; Leaflet
 + React Leaflet with a compliant OSM-derived tile provider; Haversine distance; optional OSRM routing behi
nd a service abstraction; and @capgo/background-geolocation in a Capacitor native build with a plugin versi
on matching the Capacitor major. Browser-only tracking must be clearly foreground-only. Never claim a close
d browser tab can track GPS. Location sharing must be opt-in, off by default, visible, revocable, and limit
ed to accepted friends. Build canonical friendship state through trusted server-side logic, implement deny-
by-default Firestore rules, and test with the Firebase Emulator Suite. Do not fabricate coordinates, expose
 location through user search, or store location history by default. Build a calm premium mobile-first UI a
ccording to the design system. After each phase, run build/tests, report changed files, remaining issues, a
nd manual test steps. Do not leave placeholder buttons, TODO-only core features, or insecure test rules. Do
cument limitations including Android force-stop, OS battery management, and network interruptions.
```

17. Official reference links

Capgo plugin README: https://github.com/Cap-go/capacitor-background-geolocation

Capgo setup guide: https://capgo.app/docs/plugins/background-geolocation/getting-started/

Firebase Firestore Security Rules: https://firebase.google.com/docs/firestore/security/get-started

React Leaflet: https://react-leaflet.js.org/

OpenStreetMap tile usage policy: https://operations.osmfoundation.org/policies/tiles/

OSRM API: https://project-osrm.org/docs/v5.24.0/api/